

Simulated Annealing – Mathematical Formulation

Gati Shakti Vishwavidyalaya

1 What SA Does

SA starts from the greedy solution and iteratively applies small random moves, accepting improvements always and worse solutions with decreasing probability. *Code: `src/algorithms/simulated_annealing.py`, function `solve_simulated_annealing()`*

2 Acceptance Rule (Metropolis Criterion)

$$P(\text{accept worse move}) = e^{-\Delta C/T_k}$$

Where $\Delta C = C_{\text{new}} - C_{\text{current}} > 0$ and T_k is current temperature. *Used in: `solve_simulated_annealing()` acceptance block*

3 Cooling Schedule

$$T_{k+1} = T_k \cdot r$$

Parameters in code: $T_0 = 500$, $r = 0.995$, $T_{\min} = 0.5$, 40 iterations per temperature. *Defined as: `initial_temp`, `cooling_rate`, `min_temp` in `solve_simulated_annealing()`*

4 Neighbourhood Moves

Three moves applied randomly at each iteration:

Move	What it does	Code function
Relocate	Move one trip from unit u_1 to unit u_2	<code>move_relocate()</code>
Swap	Exchange one trip between two units	<code>move_swap()</code>
Couple/Decouple	Add or remove a second unit on a trip	<code>move_couple_decouple()</code>

5 Objective

Same cost function as Greedy:

$$\text{Cost} = w_1 D_{\text{head}} + w_2 N_{\text{units}} + w_3 V_{\text{maint}} + w_4 P_{\text{uncov}} + w_5 V_{\text{time}}$$

Computed by: `compute_cost()` after every accepted move

6 Result in Our Experiment

SA reduced cost from 5,696.9 (greedy) to 4,074.1 – a 28.5% improvement. Zero timing violations. This is the best classical result. SA is analogous to classical annealing; QAOA in the paper is its quantum equivalent.